

Hands-On Development Manager Take-Home

Incremental Sync + Data Sharing

Caseware stores operational customer and case data in transactional systems, but other internal and external consumers need timely, reliable access to changes without reading directly from the source database.

In this exercise, a customer represents an organization or account in the Caseware platform. A case represents a customer-related work item such as a support issue, compliance review, onboarding investigation, billing dispute, fraud check, or audit-related follow-up. Cases are updated over time as their status changes and as new work is created.

Your task is to build a prototype of an incremental data-sharing service that:

- Reads customer and case changes from Postgres.
- publishes those changes into lake-style outputs for downstream analytics and replay.
- produces a consumer-facing incremental change feed that external systems could poll or ingest.
- and maintains checkpoints safely so no committed changes are skipped or duplicated due to timestamp ties or partial failures.

You should assume downstream consumers may use this data for use cases such as:

- syncing customer and case changes into analytics platforms.
- driving operational dashboards and reporting.
- feeding external compliance or reconciliation workflows.
- and incrementally pulling the latest batch of case updates into other tools.

The key goal is to demonstrate correctness and judgment around incremental ingestion, deterministic ordering, replay safety, and practical crash-consistent storage.

AI coding agents/tools are allowed and expected. We will evaluate how you use them, including verification, guardrails, and tradeoff handling.

0) Deliverables

Submit a repo that includes:

Required project assets

- docker-compose.yml that starts Postgres
- db/init.sql with schema and deterministic seed data
- db/changes.sql with deterministic changes applied after the first ingest to validate incrementality
- Your service implementation in any language/framework

Required docs

- README.md

Optional docs

- ARCHITECTURE_AWS.md (optional, can be done during the live interview, max 1 page, see section 4)
- AI_USAGE.md (can be explained during the live interview)

AI_USAGE.md is optional bonus credit. If you include it, we will use it to evaluate agent/tool usage maturity, verification discipline, and tradeoff handling.

1) Database contract

You will create a local Postgres DB via Docker Compose.

Postgres runtime contract

Use exactly:

- Image/version: postgres:16
- Exposed port: 5432

Environment:

- POSTGRES_DB=interop
- POSTGRES_USER=interop
- POSTGRES_PASSWORD=interop

Schema contract

Create exactly these tables, columns, and types.

customers

- customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
- name TEXT NOT NULL
- email TEXT NOT NULL UNIQUE
- country TEXT NOT NULL
- updated_at TIMESTAMPTZ NOT NULL

cases

- case_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
- customer_id BIGINT NOT NULL REFERENCES customers(customer_id)
- title TEXT NOT NULL
- description TEXT NOT NULL
- status TEXT NOT NULL
- updated_at TIMESTAMPTZ NOT NULL

Required indexes

- Index on customers(updated_at)
- Index on cases(updated_at)
- Index on cases(customer_id)

Seed data requirements for db/init.sql

Seed data must be deterministic.

Rules:

- Do not use randomness.
- Use a fixed timestamp anchor literal in SQL, then derive row timestamps from it using interval arithmetic.
- Seed at least 30 customers.
- Seed at least 200 cases.
- updated_at values must span a 30-day window relative to that fixed anchor.

Case text should include varied keywords so downstream artifacts are meaningful, for example:

- billing
- audit
- compliance
- payments
- reconciliation
- onboarding
- fraud
- AML

Change script requirements for db/changes.sql

This script will be applied between two ingestion runs.

It must deterministically simulate incremental changes using a fixed timestamp literal that is later than every timestamp used in db/init.sql.

It must:

- Update exactly 5 existing cases:
 - set updated_at to the fixed change timestamp
 - change their status
- Insert exactly 2 new customers with updated_at set to the fixed change timestamp
- Insert exactly 10 new cases linked to a mix of existing and new customers, with updated_at set to the fixed change timestamp

2) Service requirements

Build a small service with one required endpoint and local filesystem outputs.

2.1 Incremental ingestion + checkpoint correctness

Checkpoint contract

Store ingestion state locally at:

- ./state/checkpoint.json

The checkpoint must be per-table and must use a composite watermark, not a timestamp alone.

Required shape:

```
{ "customers": { "updated_at": "2026-03-31T12:00:00Z", "last_pk": 30 }, "cases": { "updated_at": "2026-03-31T12:00:00Z", "last_pk": 200 } }
```

last_pk means:

- customer_id for customers
- case_id for cases

If no checkpoint exists, treat that as an initial empty watermark.

Required incremental query semantics

For each table, ingest rows where:

- updated_at > checkpoint.updated_at
- OR updated_at = checkpoint.updated_at AND primary_key > checkpoint.last_pk

Rows must be processed in deterministic order:

- updated_at ASC
- primary_key ASC

This composite watermark requirement exists to avoid missing rows that share the same timestamp.

Source consistency assumption

Concurrent source mutations during /ingest are out of scope for this exercise.

You may assume:

- the source Postgres data does not change while a single ingest run is executing
- you do not need to implement transaction snapshotting or cross-table source consistency guarantees beyond this assumption

Endpoint

POST /ingest?dry_run=true|false

Behavior

The endpoint must incrementally ingest rows from both tables using the checkpoint rules above.

If dry_run=true:

- Compute deltas
- Compute the manifest
- Do not write lake outputs
- Do not write durable events
- Do not write share artifacts
- Do not update the checkpoint file

If dry_run=false:

- Write lake outputs
- Write durable events
- Write share artifacts
- Return a manifest
- Advance the checkpoint only if the run succeeds end-to-end

Dry-run checkpoint semantics

The manifest must always include both:

- checkpoint_before
- checkpoint_after

For `dry_run=true`, `checkpoint_after` means the predicted checkpoint that would be committed if the same batch were executed successfully with `dry_run=false`.

Important:

- `checkpoint_after` in the response is informational only during dry run
- the on-disk `./state/checkpoint.json` must remain unchanged during dry run

Failure safety rule

Checkpoint advancement must follow a crash-consistent local strategy with respect to the durable local outputs of a successful run.

The goal is that a process crash or failure must not leave the system in a state where:

- the checkpoint has advanced
- but the corresponding durable outputs are missing or partially committed

For this exercise, the crash-consistency boundary includes:

- touched lake files
- durable event file
- share artifact file write/replace
- checkpoint file

The following are not part of the crash-consistency boundary:

- stdout logging
- the HTTP response itself

A practical local approach is acceptable, for example:

- stage files in a temp run directory
- write temp files fully
- atomically rename or replace final files where possible
- only then replace `./state/checkpoint.json`

You are not required to implement true multi-file atomic commit semantics across the filesystem. We will evaluate whether your approach is crash-consistent and prevents checkpoint advancement ahead of durable outputs.

2.2 Deterministic run_id

`run_id` must be deterministic from inputs so that the same source DB state and same checkpoint produce the same `run_id`.

A simple acceptable strategy is:

- build a canonical JSON object containing:
 - `checkpoint_before`
 - for each table, the ordered delta row identities as selected by the incremental query
- each row identity should include:
 - table name
 - primary key
 - `updated_at`
- serialize with stable key ordering
- hash it using SHA-256
- use the hex digest, or a stable prefix of it, as `run_id`

Because the row identities are already ordered by:

- updated_at ASC
- primary_key ASC

the resulting run_id must be deterministic for the same source state and checkpoint.

2.3 Local lake output (JSONL)

Write valid JSONL files to the local filesystem.

Required layout:

- ./lake/customers/date=YYYY-MM-DD/data.jsonl
- ./lake/cases/date=YYYY-MM-DD/data.jsonl

Partitioning rule

YYYY-MM-DD is the UTC calendar date derived from each row's updated_at.

If a single ingest run touches multiple dates, you must write multiple partitions.

CDC rule

For this exercise, the lake is an append-only CDC-style change log, not a current-state snapshot.

Each successful non-dry ingest appends the changed rows for that run to the appropriate date partition or partitions, based on each changed row's updated_at.

Records written to the lake for a run must correspond exactly to the incremental delta selected by the checkpoint query for that table.

Rerunning ingest with no source DB changes must not create duplicates, because no new delta should be selected.

Deterministic ordering within each batch must follow:

- updated_at ASC
- primary_key ASC

A simple acceptable strategy is:

- determine the incremental delta for each table
- group changed rows by UTC updated_at date
- append those changed rows as JSONL records to the appropriate partition file or files
- ensure a no-op run writes nothing

2.4 Manifest returned by /ingest

Return a JSON manifest in the response body with at least:

- run_id as a string
- started_at
- finished_at
- dry_run
- checkpoint_before
- checkpoint_after

For each table include:

- delta_row_count
- lake_paths
- share_path
- schema_fingerprint

Schema fingerprint

A hash of column names and types is sufficient.

Manifest path semantics

lake_paths should list the final target lake files for touched partitions.

lake_paths should be an empty array when delta_row_count = 0.

share_path should be the final target share artifact path when a share artifact is produced.

share_path should be null when delta_row_count = 0.

For dry_run=true, these paths are still included as predicted output targets, but no files may be written.

2.5 Delta events

After a successful non-dry ingest, produce one event per table.

Durable event requirement

Write per-run events to:

- ./events/<run_id>.jsonl

This file must contain one valid JSON line per table included in the run.

Each event must include at least:

- table
- run_id
- schema_fingerprint
- delta_row_count
- lake_paths
- share_path
- checkpoint_after

Because this is a per-run file, it may be safely staged and atomically moved into place as part of the run commit.

Stdout requirement

Also print one event per table to stdout as a best-effort mirror of the durable event.

Important:

- stdout emission is not part of the crash-consistency boundary
- a successful run must not be considered failed only because stdout logging was interrupted

2.6 Consumer-facing incremental share artifact

After a successful non-dry ingest, produce a deterministic consumer-facing incremental share artifact per table.

Required output

Write share artifacts to:

- ./share/customers/changes.jsonl
- ./share/cases/changes.jsonl

These files represent the latest successful incremental batch that an external consumer would read after a successful ingest.

They are not a full snapshot.

They are a deterministic companion change feed for downstream synchronization.

Required contents

Each record must be valid JSON and include at least:

- table
- op
- primary key field:
 - o customer_id for customers
 - o case_id for cases
- updated_at
- run_id
- schema_fingerprint
- checkpoint_after
- record

Where:

- op may be upsert
- record contains the full row payload for that changed row

Determinism rule

Given the same source data and the same checkpoint, the share artifact contents and record order must be byte-for-byte identical across repeated successful runs.

Because run_id is deterministic from inputs, including it in the artifact does not break this requirement.

Ordering rule

Records in each share artifact must be ordered by:

- updated_at ASC
- primary_key ASC

Zero-delta rule

If a successful non-dry run has delta_row_count = 0 for a table:

- do not create a new share artifact for that table
- do not modify the existing share artifact for that table
- report share_path = null for that table in the manifest and durable event

This requirement exists so that no-op runs do not create misleading or duplicate share batches.

Dry-run rule

If dry_run=true:

- do not write share artifacts

Relationship to events

The durable event written to ./events/<run_id>.jsonl should summarize the share artifact produced for each table.

If a table has zero delta:

- the event should still exist for that table
- delta_row_count should be 0

- share_path should be null

Documentation note

Briefly document in README.md:

- replay strategy for consumers
- idempotency expectations
- how schema evolution would be handled in a production version

3) Documentation requirements

3.1 README.md

Include:

- prerequisites, including Docker
- how to run the DB and service
- example call to /ingest
- example outputs for lake files, share artifacts, and events
- important assumptions
- a short note describing your local crash-consistency strategy
- a short note describing replay/idempotency expectations for downstream consumers
- a short note on how schema evolution would be approached in production

3.2 AI_USAGE.md (bonus)

Optional bonus credit.

If included, document:

- tools/agents used
- key prompts or transcript excerpts
- what you verified manually
- commands you ran
- edge cases you checked
- what the agent got wrong and how you corrected it

4) ARCHITECTURE_AWS.md (optional, can be done during live interview session, max 1 page)

This is an optional documentation-only design exercise.

Important clarification

Do not model this literally as “the same local /ingest endpoint running on AWS.”

We want a production ingestion and sharing architecture for the same problem domain:

- incremental customer/case sync

- durable lake storage
- event-driven propagation
- consumer-facing incremental data sharing using open table standards or companion change feeds

Your design should describe a production-grade AWS ingestion architecture and consumer-serving architecture, not a literal lift-and-shift of the local toy service.

Not required

- no AWS account access
- no Terraform, CDK, or CloudFormation
- no deployed AWS resources
- no real S3, SQS, Athena, Glue, or Iceberg implementation in code
- no AWS Console screenshots

What we expect if you choose to include it

A clear 1-page architecture description with:

- diagram
- service choices
- reasoning
- NFR coverage
- failure handling
- replay/idempotency strategy

Assume these production NFRs:

Multi-tenant SaaS

- about 5,000 tenants

Data size

- 50M customers
- 200M cases

Change rate

- cases: up to 5,000 updates/sec peak, 1,000/sec sustained
- customers: up to 500 updates/sec peak

Freshness SLA

- new or updated rows must be reflected in the lake and available to downstream consumers within 10 minutes at p95

Consumer/query load

These requirements apply to the production consumer-serving interface that exposes incremental shared data to downstream consumers, not necessarily to direct queries against raw lake storage.

- 300 QPS steady
- burst to 1,000 QPS
- p95 latency under 300ms for common incremental access patterns
- p99 latency under 800ms

Availability

- ingest and consumer-facing APIs: 99.9% monthly

Recovery targets

- RPO $\leq$ 5 minutes
- RTO $\leq$ 30 minutes for APIs

Lake durability

- treat as effectively zero data loss
- describe versioning and immutability strategy

Cost

Identify the top 3 cost drivers and how you would control them over time.

What to include in the page

- a high-level diagram in Mermaid or ASCII
- the data flow from source-of-truth changes into:
 - durable storage
 - eventing
 - external consumption/shared-data serving
- service choices with brief rationale
- reliability and replay strategy
- idempotency strategy
- tradeoffs and assumptions in 3 to 6 short bullets

5) EXECUTION_PLAN.md (max 1 page)

How you'd deliver the AWS production version with a small team:

- milestones (e.g., 2–3 sprints)
- work breakdown + sequencing
- what you'd delegate vs do yourself
- quality gates (CI checks, tests, rollout strategy)
- top risks + mitigations

6) Evaluation rubric (weighted)

Hands-on implementation (60%)

- correctness of incremental ingest + checkpoint safety
- idempotency and failure handling
- clean structure and pragmatic design
- tests that validate behavior

Hands-on manager signal (40%)

- production-ready AWS architecture reasoning (If not provided initially otherwise we can do it on a live interview session)
- execution plan quality (scope, sequencing, risk management)
- AI usage discipline (verification, guardrails)
- review checklist practicality

7) Notes and assumptions

You may use any language, framework, or test stack.

Reasonable simplifications are acceptable if they are:

- explicit
- documented
- consistent with the required contracts